AI Engineering Prompt Playbook

Standard Operating Templates for High-Accuracy Code & Architecture Generation. Built specifically to eliminate model drift, secure deterministic outputs, and standardize tech workflows.

TARGET: SOFTWARE ENGINEERING & ARCHITECTURE TEAMS

FRAMEWORK: SYSTEM PROMPT STANDARD

Engineering-Grade Foundations

To maximize accuracy and consistency when leveraging LLMs, templates must rigidly lock down context layers.

Standardizing prompts eliminates structural assumptions, blocks legacy coding anti-patterns, and ensures secure, compile-ready codebase modules.

Prevents silent instruction leakage and parser confusion.

The 5 Core Anchor Pillars

- → **Role Definition:** Narrows probability token boundaries immediately.
- → **Domain Context:** Restricts technical answers to current environment scope.
- → **Detailed Specifications:** Explicitly maps features to build.
- → **Strict Constraints:** Defines compilation guardrails and negative bounds.
- → **Expected Output Structure:** Declares clean directory layouts and format keys.

Template 1: Full-Scale Architecture

Execution Context

Use this structure when bootstrapping a new microservice, library, or standalone module from scratch.

It forces the target model to map directory trees, configuration footprints, and domain classes concurrently.

→ Saves up to 3 hours of initial setup.

→ Locks dependencies to declared target versions.

BOOTSTRAPPING_PROMPT_PART1.TXT

```
### Role & Objective
Act as an expert Java software architect. Generate a clean,
production-ready Java project based on the specs below. Adhere to
modern Java best practices and robust error handling.

### Project Overview
* Project Name: [e.g., Inventory Management System]
* Java Version: [e.g., Java 17 / Java 21]
* Build Tool: [e.g., Maven / Gradle]
* Libraries: [e.g., Spring Boot 3.x, Spring Data JPA, JPA]

### Core Specifications & Features
1. [Feature 1: REST API with validation]
2. [Feature 2: Database persistence specs]
3. [Feature 3: Custom exception handler]
```

Enforcing Strict Code Guardrails

BOOTSTRAPPING_PROMPT_PART2.TXT

```
### Strict Rules & Constraints
* Architecture: Layered (Controller, Service, Repository)
* ORM Rules: Proper transactional scopes on Services.
* Code Quality: Constructor injection over field-level @Autowired.
Reduce boilerplate with Lombok.
* Testing: Unit test suites using Mockito.
* What to Avoid: Do not use field injection. No hardcoded configs;
use application.properties.

### Expected Output Format
Provide the response in structured order:
1. **Directory Tree:** Visual directory breakdown.
2. **Configurations:** Complete pom.xml & properties.
3. **Source Code:** Fully written class files.
4. **Brief Explanation:** Component flow summary.
```

Why This Prevents Failures

By specifying both target guidelines and **"What to Avoid"** constraints directly in the instructions, you prevent common anti-patterns before generation starts.

⚠ Never rely on the AI to assume code preferences (like injection types or testing tools). Enforce it in the constraints.

Template 2: Daily Task Framework

The Day-to-Day Skeleton

A modular framework engineered for rapid developer workflows like legacy code refactoring, query optimization, bug fixing, or creating utility classes.

- → Provides tight boundaries for isolated engineering tasks.
- → Avoids conversational fluff, chat introductions, and verbose explanations.

DAILY_TASK_FRAMEWORK.TXT

```
### 1. Role & Context
* Role: Act as an expert [e.g. React/AWS] Senior Engineer.
* Context: Working on an [e.g. e-commerce service] using [e.g. Spring Boot, PostgreSQL].

### 2. The Task
I need you to [Core action verb: Create / Debug / Refactor / Optimize] the following:
> [Describe exact task goal, e.g. Create a custom SQL query using CTE to fetch hierarchical data.]

### 3. Inputs & Existing Code
[Paste relevant code, errors, or schemas here]
```

Enforcing Task Specifications

DAILY_TASK_CONSTRAINTS.TXT

```
### 4. Strict Constraints & Rules
* Standards: [e.g. Follow SOLID, thread safety]
* Performance/Security: [e.g. Avoid N+1 queries, sanitize all SQL
variables]
* What to AVOID: [e.g. No deprecated libraries, do not write nested
loops]

### 5. Expected Output
Please provide:
1. **The Solution:** Optimized, production-ready code with concise
inline notes on *why* choices were made.
2. **Edge Cases Handled:** Clear bulleted list.
3. **Verification:** [e.g. Sample JUnit test / Curl testing command]
```

Output Guarantees

By splitting instructions, constraints, and target code outputs, the model guarantees the delivery of highly focused, tested logic blocks.

⚡ Speeds up integration times because the code is structured explicitly to fit into your active files.

Daily Execution Guide

A. Debugging

"Analyze the attached stack trace and locate the root cause. Provide the exact code fix and explanation of why the failure occurred."

B. Refactoring

"Refactor the legacy method to improve readability and dry compliance without altering the public API method signature."

C. Optimization

"Optimize the attached SQL query. It is executing slow full-table scans. Provide target indices and rewrite logic."

⭐ THE GOLDEN AI ENGINEERING RULE

"Never let the AI assume context. If a rule or constraint matters to your architecture (e.g. thread safety, compliance, memory footprint), explicitly list it in the Strict Constraints block or under What to Avoid."